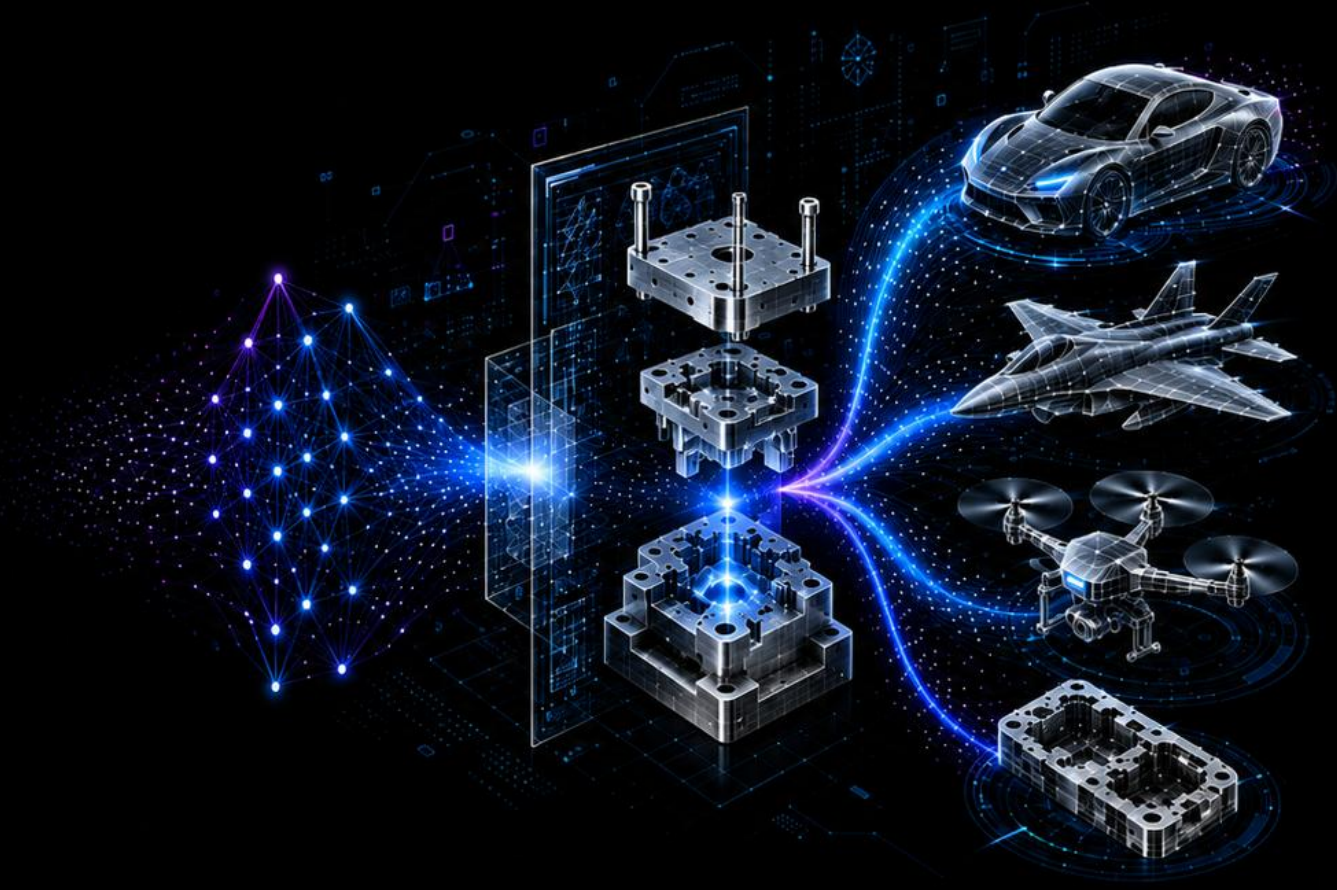
7장 RNN

스터디 보고



정승한
(지도교수: 양선웅)

한양대학교 ERICA 기계공학과

Intelligent Design for Engineering Applications Lab

RNN : Recurrent Neural Network

Intelligent Design for Engineering Applications Lab

시간에 따라 변하는 데이터는 변동이 존재

기존 ANN은 입력 데이터만을 다루며, 이는 순서를 제대로 이해하기 어려움
주로 시계열 분석 (the components of time series)에서 사용됨

변동

❖ 불규칙 변동

- 규칙성 없이 예측 불가능한, 우연적으로 발생하는 변동

❖ 추세 변동

- 장시간에 걸쳐 지속적으로 증가, 감소
- 단기간에는 추세 변동을 찾기 힘듦

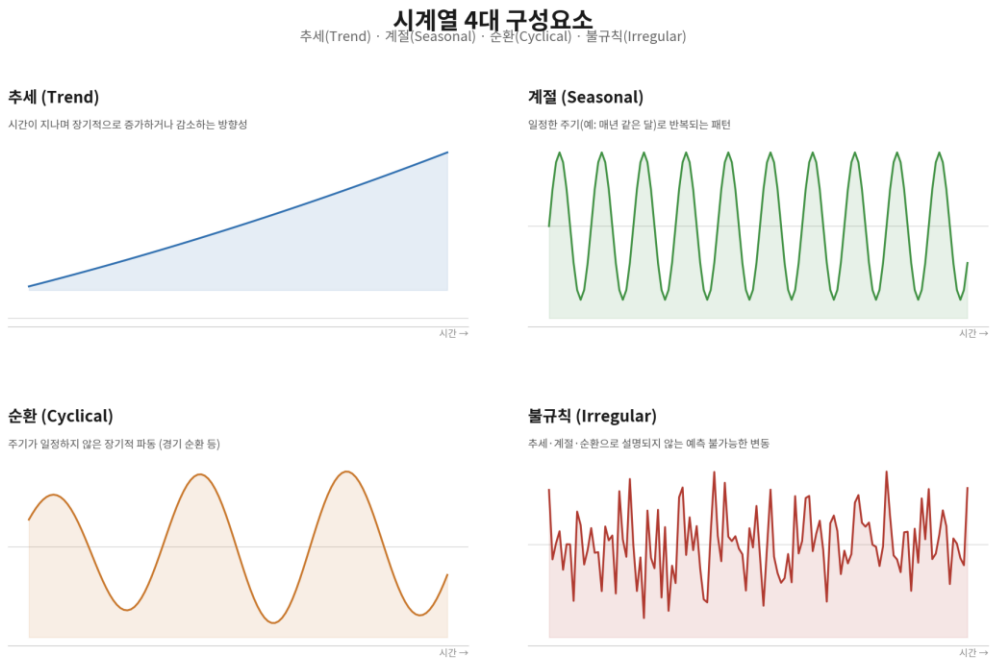
❖ 순환 변동

- 일정한 주기 (2~3년)로 순환적인 변동

❖ 계절 변동

- 계절 또는 사회적 관습에 따라 1년 주기로 발생하는 변동

Figure



RNN : Recurrent Neural Network

Intelligent Design for Engineering Applications Lab

시간을 독립변수로 종속변수를 예측하는 4가지 모델 소개

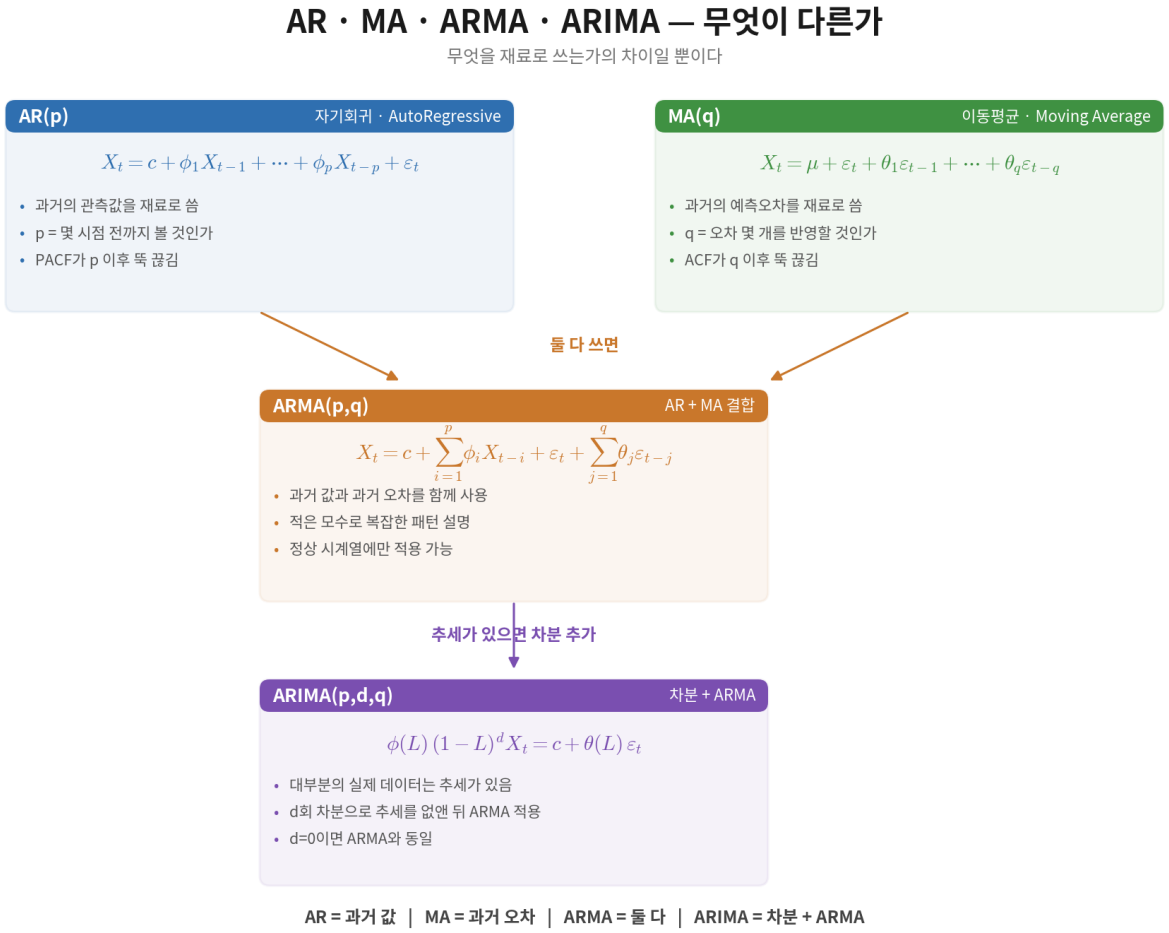
model

❖ AR
- 이전 관측값이 이후 관측값에 영향을 미침
- $Z_t = \phi_1 Z_{t-1} + \phi_2 Z_{t-2} + \cdots + \phi_p Z_{t-p} + a_t$

❖ MA
- 과거의 오차가 현재의 오차에 영향을 미침
- $Z_t = \theta_1 a_{t-1} + \theta_2 a_{t-2} + \cdots + \theta_p a_{t-p} + a_t$

❖ ARMA
- AR과 MA를 합한 형태의 수식을 사용
- 과거의 관측 값과 오차를 반영

❖ ARIMA
- AR과 MA를 반영하고, 시계열 데이터를 안정화 시키기 위해 차분 (현재 값 – 이전 값) I를 진행
- 각각 p,q,d로 횟수를 표기



RNN : Recurrent Neural Network

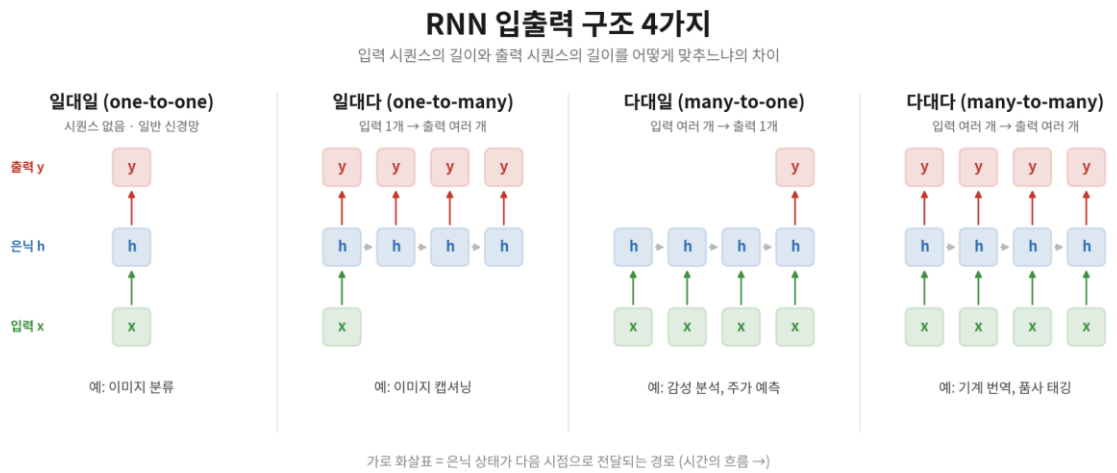
RNN의 기본 구조

RNN은 문제에 따라 입출력 구조가 달라짐

입출력 구조

- ❖ 일대일 구조
  - x에서 y로
  - 이미지 분류, 표 형식 데이터 분류 (일반적인 ANN과 동일)
- ❖ 일대다 구조
  - 이미지 생성, 음악 생성, 초기조건을 이용한 시계열 예측
  - x에서 y1, y2, y3…
- ❖ 다대일 구조
  - 여러 개의 입력을 순서대로 받아 하나의 출력 생성
  - 시계열 이상 탐지, 스팸 메일 분류 등
  - x1, x2…에서 y로
- ❖ 다대다 구조
  - 입력과 출력이 같은 경우 각 시점마다 출력이 존재
  - 입력과 출력이 다른 경우 입력을 모두 읽고 새로운 출력 생성 (Encoder – Decoder 구조)

Figure



VAE와 GAN (부록)

- ❖ VAE와 GAN은 기존 Encoder – Decoder 구조와 유사한 면 존재
- ❖ E – D 구조는 기계번역과 시퀀스 변환에 활용되며, 이후 다양한 생성 모델에서도 잠재 표현과 데이터 생성 개념으로 확장됨

RNN : Recurrent Neural Network

Intelligent Design for Engineering Applications Lab

시간을 독립변수로 종속변수를 예측하는 4가지 모델 소개

RNN 구조

❖ 기본적인 RNN 셀

- $\boldsymbol{h}_t = tanh(\boldsymbol{W}_{xh}\boldsymbol{x}_t + \boldsymbol{W}_{hh}\boldsymbol{h}_{t-1} + \boldsymbol{b}_h)$

❖ weight sum 계산

- $\boldsymbol{a}_t = \boldsymbol{W}_{xh}\boldsymbol{x}_t + \boldsymbol{W}_{hh}\boldsymbol{h}_{t-1} + \boldsymbol{b}_h$

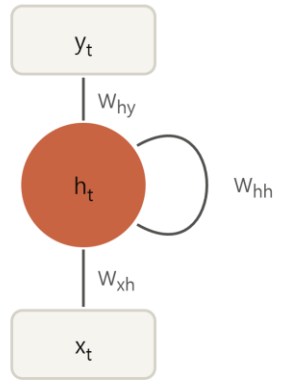
❖ 이후 활성화 함수로 하이퍼볼릭 탄젠트 사용

- 기본 RNN에서 은닉상태 활성화 함수로 주로 tanh 사용
- 출력 범위가 −1 ~ 1 이므로 은닉상태 값의 크기를 제한하고, 양수와 음수를 모두 표현할 수 있다는 장점

❖ sigmoid는?

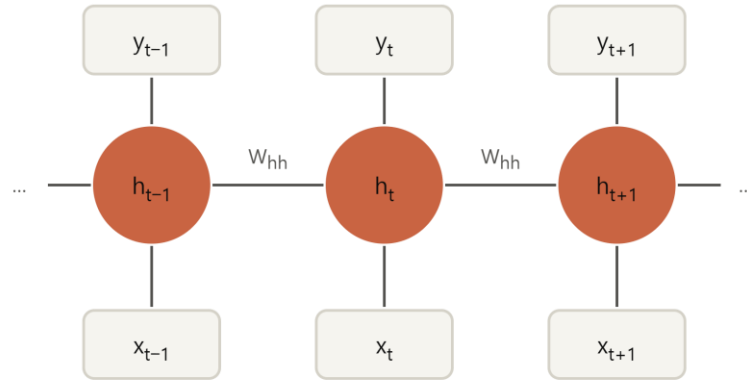
- Sigmoid는 0 ~ 1의 출력을 가지는데, 출력 범위가 0이 중심이 아니므로 잘 사용하지 않음

접힌 형태



은닉상태가 자기 자신으로 되돌아오는 순환 연경

시간축으로 펼친 형태



매 시점이 똑같은 가중치를 공유하는 깊은 신경망 (가중치 공유 · weight sharing)

$$h_t = tanh(\, W_{hh}\, h_{t-1} + W_{xh}\, x_t + b_h \,)$$ ← 이전 기억 + 현재 입력을 섞어 새 기억을 만든다

$$y_t = W_{hy}\, h_t + b_y$$ ← 그 기억에서 이번 시점의 출력을 뽑는다

W_{xh}, W_{hh}, W_{hy} 는 모든 시점에서 동일 → 파라미터 수가 시퀀스 길이와 무관하고, 길이가 달라도 같은 모델을 쓸 수 있다

RNN : Recurrent Neural Network

Intelligent Design for Engineering Applications Lab

RNN은 가중치를 공유함

RNN의 출력

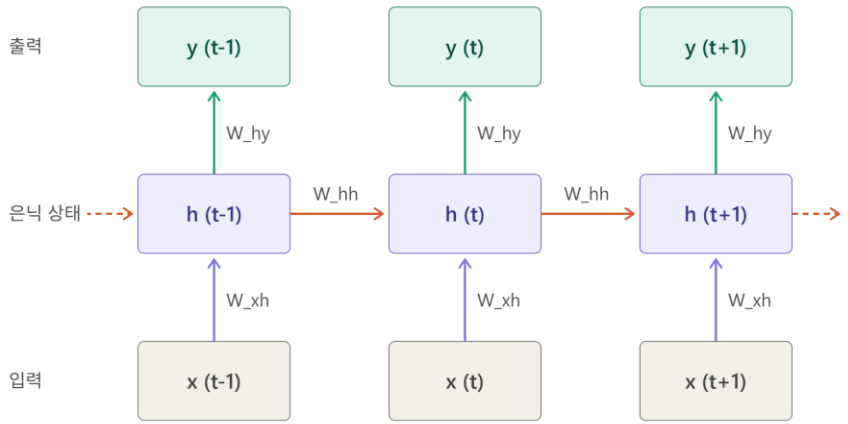
- 현재 은닉 상태를 이용해 출력을 계산
- $o_t = W_{hy}h_t + b_y$

RNN의 가중치

- 문장을 처리할 때의 경우, 단어의 위치에 따라 서로 다른 규칙을 사용할 필요는 없음
- t = 1,2,3.. 마다 동일한 가중치 행렬을 사용
- RNN은 시계열 데이터를 처리할 때, 시간 위치가 달라지더라도 같은 종류의 패턴을 같은 방식으로 해석
- 만약 가중치를 다르게 사용한다면 의미나 패턴을 학습하기보다, 특정 위치에 과도하게 의존하는 문제 발생 가능
- 또한 같은 가중치를 사용하므로 파라미터 수가 크게 줄어듦

각 가중치의 의미

- 입력 가중치 W_{hy}
  - 현재 입력 x에서 어떤 특징을 추출할지를 결정
- 순환 가중치 W_{hy}
  - 이전 시점의 정보 중 어떤 정보를 현재 시점으로 전달할지 결정
- 출력 가중치 W_{hy}
  - 현재 은닉 상태를 실제 출력 값으로 변환

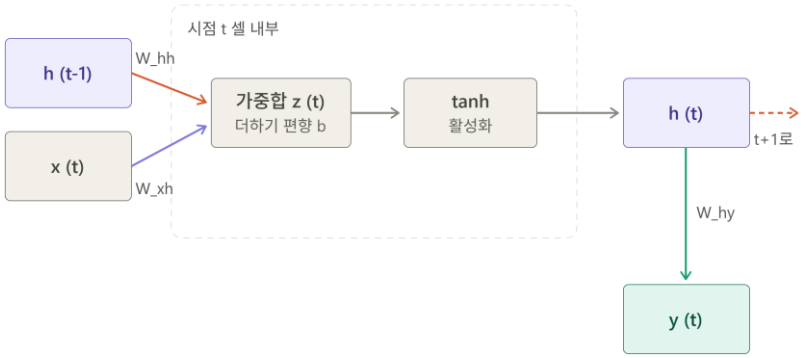


RNN : Recurrent Neural Network

RNN의 Forward Propagation, BPTT (Backpropagation Through Time)

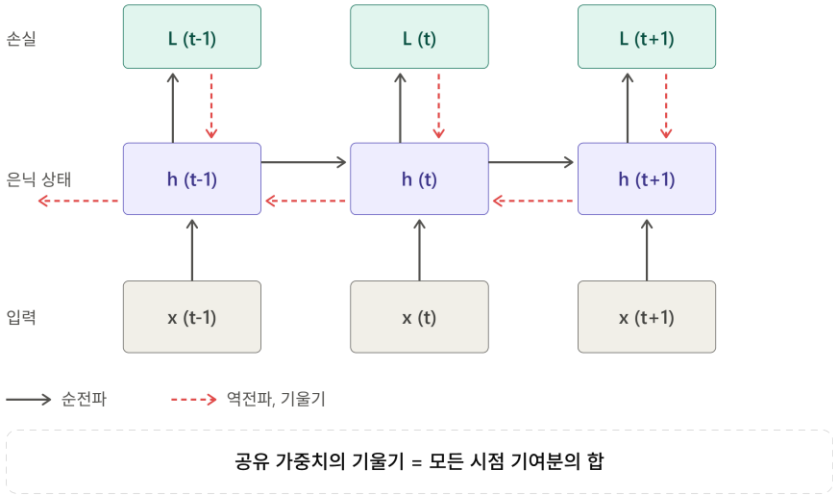
RNN 순전파

- 입력 데이터를 시간 순서대로 받아, 이전 시점의 정보를 현재 시점으로 전달하며 은닉 상태를 계산하는 과정
- 일반적인 신경망은 입력을 한번에, RNN은 순차적으로 처리
- x_1 -> x_2 -> x_3 -> … -> x_T
- $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$
- t = 0 에서 h는 0벡터



BPTT

- 시간을 통한 역전파
- Loss에서 시작하여 반대 방향으로 미분
- $\frac{\partial L}{\partial h_k} = \frac{\partial L}{\partial h_T}\frac{\partial h_T}{\partial h_{T-1}}\frac{\partial h_{T-1}}{\partial h_{T-2}} \cdots \frac{\partial h_{k+1}}{\partial h_k}$
- $\prod_{t=k+1}^{T} \frac{dh_t}{dh_{t-1}}$
- 이러한 반복 곱으로 인해 기울기 소실과 기울기 폭주가 발생 가능



RNN : Recurrent Neural Network

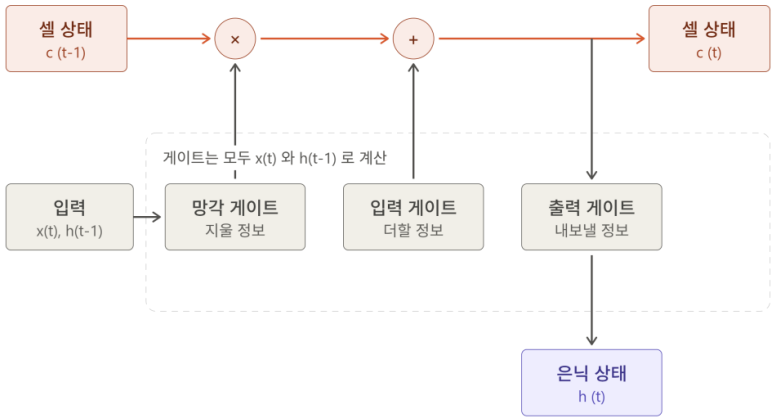
기본 RNN의 한계 -> LSTM 등장

한계점

- ❖ 장기 기억이 어려움
  - 기울기 소실 때문에 먼 과거의 정보가 희석됨
- ❖ 순차 계산이 필요하다
  - 현재 시점을 계산하려면 이전의 은닉층의 계산이 필수적
- ❖ 긴 시퀀스에서 계산 량이 증가
  - 시계열이 길어지면 순전파와 역전파 계산량이 너무 커짐
- ❖ 정보를 하나의 은닉 상태에 계속 압축
  - 시퀀스가 길어질수록 초기 정보가 희석되거나 덮어 쓰이는 문제

LSTM

- ❖ LSTM : Long short – Term Memory
- ❖ LSTM은 일반 은닉 상태뿐 아니라 셀 상태 사용
- ❖ LSTM은 게이트를 이용해 다음을 결정
  - 과거 정보 중 무엇을 잊을 것인지
  - 현재 입력 중 무엇을 저장할 것인지
  - 저장된 정보 중 무엇을 출력할 것인지



RNN : Recurrent Neural Network

LSTM의 개념과 gate

LSTM

- ❖ 긴 시간 동안 필요한 정보를 기억하도록 만든 RNN의 개선형 구조
- ❖ Cell state와 forget, input, output gate 개념 출현
- ❖ 셀 상태 (Cell state)
  - Cell state는 시퀀스 전체를 따라 흐르는 장기 기억
  - 게이트에 의해 일부 정보가 삭제되거나 추가되는 형식
  - LSTM의 장기 의존성처리에 강한 이유

LSTM의 게이트

- ❖ Gate란?
  - 정보를 통과시키거나 차단하는 장치 (0~1사이)
  - 0과 가까우면 정보를 차단, 1에 가까우면 통과
  - 현재 입력 x와 이전 은닉 상태 h, 이전 셀 상태 C를 사용
  - LSTM에서는 sigmoid 함수가 주로 사용

Forget 게이트

- ❖ 수식
  - $f_t = \sigma(\boldsymbol{W}_f \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_f)$
  - 위의 $\boldsymbol{f}_t$와 Cell state $\boldsymbol{C}_{t-1}$를 원소별로 곱
  - 예를 들어 $\boldsymbol{C}_{t-1} = [0.8, 0.5, -0.3]$, $\boldsymbol{f}_t = [1, 0.2, 0]$일 때 [0.8, 0.1, 0]으로 1번 기억 그대로 유지, 2번은 대부분 삭제, 3번 기억은 완전히 삭제

Input 게이트

- ❖ 수식
  - $i_t = \sigma(\boldsymbol{W}_i \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_i)$
  - 새로운 정보를 얼마나 저장하는지 결정
  - $\tilde{\boldsymbol{c}}_t = tanh(\boldsymbol{W}_c \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_c)$
  - $\tilde{\boldsymbol{c}}_t$는 현재 입력으로 만들어진 새로운 기억 후보
  - 이 둘을 $i_t \odot \tilde{\boldsymbol{c}}_t$하여 새로 저장할 정보 결정

RNN : Recurrent Neural Network

LSTM의 개념과 gate

Output 게이트

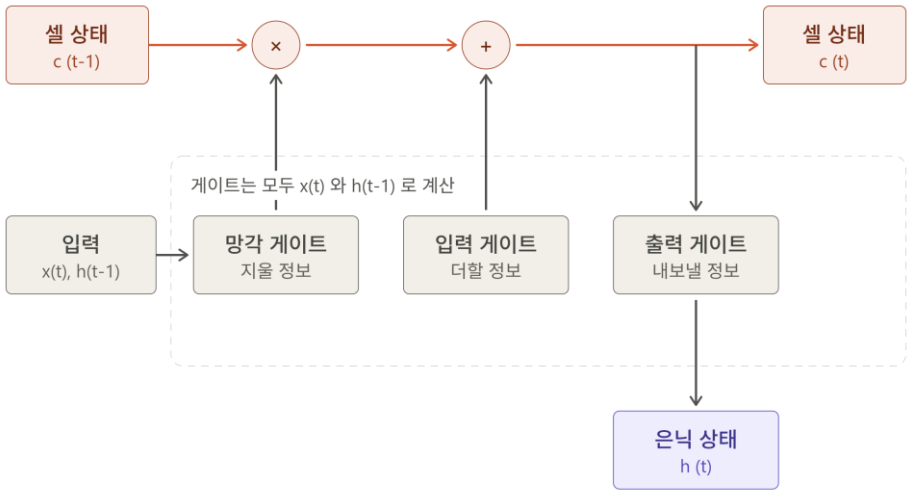
- ❖ 이전 기억에서 남길 정보와 새롭게 저장할 정보를 합쳐 현재 셀 상태 계산
- ❖ $\boldsymbol{c}_t = f_t \odot \boldsymbol{c}_{t-1} + i_t \odot \tilde{\boldsymbol{c}}_t$
- ❖ 각각 유지되는 기억, 추가되는 기억

- $o_t = \sigma(\boldsymbol{W}_o \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_o)$ (출력 게이트 값 계산)
- $tanh(\boldsymbol{c}_t)$ (현재 셀 상태 tanh 함수에 통과)
- $\boldsymbol{h}_t = o_t \odot tanh(\boldsymbol{c}_t)$
- 셀 상태 c의 모든 정보를 출력 x
- 현재 시점에 필요한 h만 내보냄
- 출력 게이트는 장기 기억 중에서 현재 시점에 필요한 정보만 은닉 상태로 전달

기울기 소실

- ❖ LSTM은 셀 상태 업데이트가 덧셈 구조를 가짐
- ❖ RNN은 시간방향 역전파할 때 가중치와 활성화 함수의 미분이 반복해서 곱
- ❖ LSTM은 $\frac{dc_t}{dc_{t-1}} \approx f_t$ 이므로 역전파시 기울기가 크기 줄어들지 않고 이전 시점까지 전달 가능 (forget 게이트로 인해)

Figure



RNN : Recurrent Neural Network

GRU : Gated Recurrent Unit

GRU의 개념

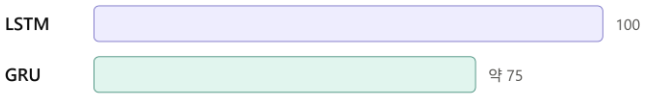
- ❖ RNN의 장기 기억 문제를 해결하기 위해 만든 게이트 기반 순환 신경망
- ❖ LSTM과 같은 목적을 가지지만, 구조는 더욱 단순
- ❖ LSTM에 비해 게이트 2개, 상태 1개로 축소됨
- ❖ GRU에는 두개의 게이트가 존재
  - Update gate
  - Reset gate
- ❖ GRU는 장기 기억과 현재 출력을 하나의 은닉 상태로 통합함

GRU vs LSTM

- ❖ LSTM은 세 개의 gate, 두 개의 state를 사용
- ❖ GRU는 두 개의 gate, 한 개의 state 사용
- ❖ 계산 속도가 빠르고 메모리 사용량이 작은 장점이 있음

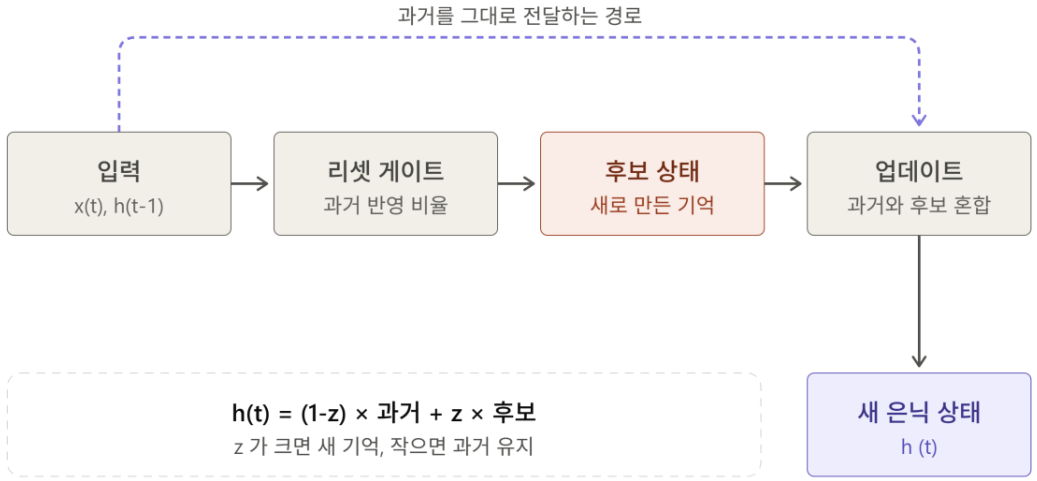
비교 표

파라미터 수, LSTM 을 100 으로 둔 상대 비교



항목	LSTM	GRU
게이트 수	3 개, 망각 입력 출력	2 개, 리셋 업데이트
상태 벡터	c 와 h 두 개	h 하나
학습 속도	상대적으로 느림	더 빠름
유리한 상황	긴 시퀀스, 큰 데이터	적은 데이터, 빠른 실험

Figure



RNN : Recurrent Neural Network

GRU : Gated Recurrent Unit

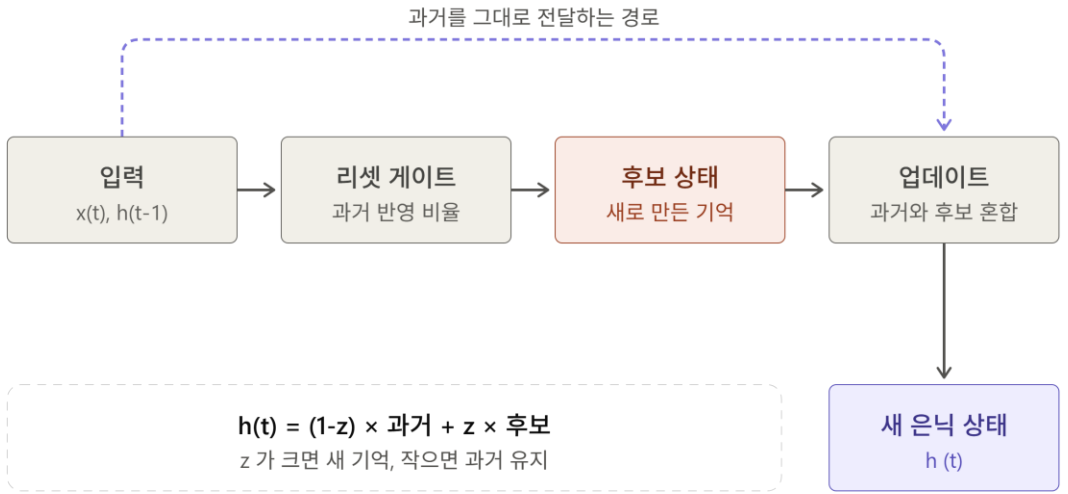
입력과 출력

- ❖ 시점 t에서 GRU는 현재 입력과 이전 은닉 상태 값을 입력으로 받음
- ❖ 그리고 현재 은닉 상태 출력 h_t

gate

- ❖ Update gate
  - 과거의 정보를 얼마나 유지하고 새로운 정보로 얼마나 바꿀 것인지?
- ❖ Reset gate
  - 새로운 후보 정보를 만들 때 과거 정보를 얼마나 참고할 것인지?

Figure



RNN : Recurrent Neural Network

GRU : Gated Recurrent Unit

GRU의 계산 순서

❖ 1단계 : update gate 계산

- $z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$
- 이전 정보와 새로운 정보의 혼합 비율 결정 (sigmoid 함수)

❖ 2단계 : reset gate 계산

- $r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$
- 새로운 기억 후보를 만들 때 얼마나 과거를 참고할지 결정

❖ 3단계 : 후보 은닉 상태 계산

- $\tilde{h_t} = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$
- 현재 입력을 바탕으로 새롭게 기억할 수 있는 정보의 후보
- 붉은색 부분은 리셋 게이트가 적용된 과거 정보

❖ 4단계 : 현재 은닉 상태 계산

- $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$
- 현재 기억 = 유지한 과거 기억 + 반영한 새로운 기억

Figure1

(1) $z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$
업데이트 게이트, 새 정보로 갈아끼울 비율

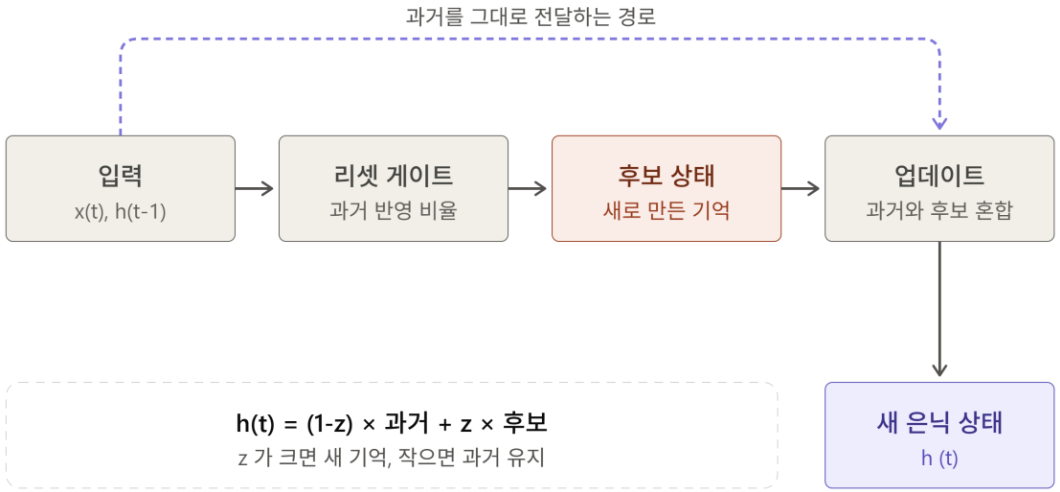
(2) $r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$
리셋 게이트, 과거를 얼마나 참고할지

(3) $\tilde{h_t} = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$
후보 상태, 리셋된 과거와 현재 입력으로 생성

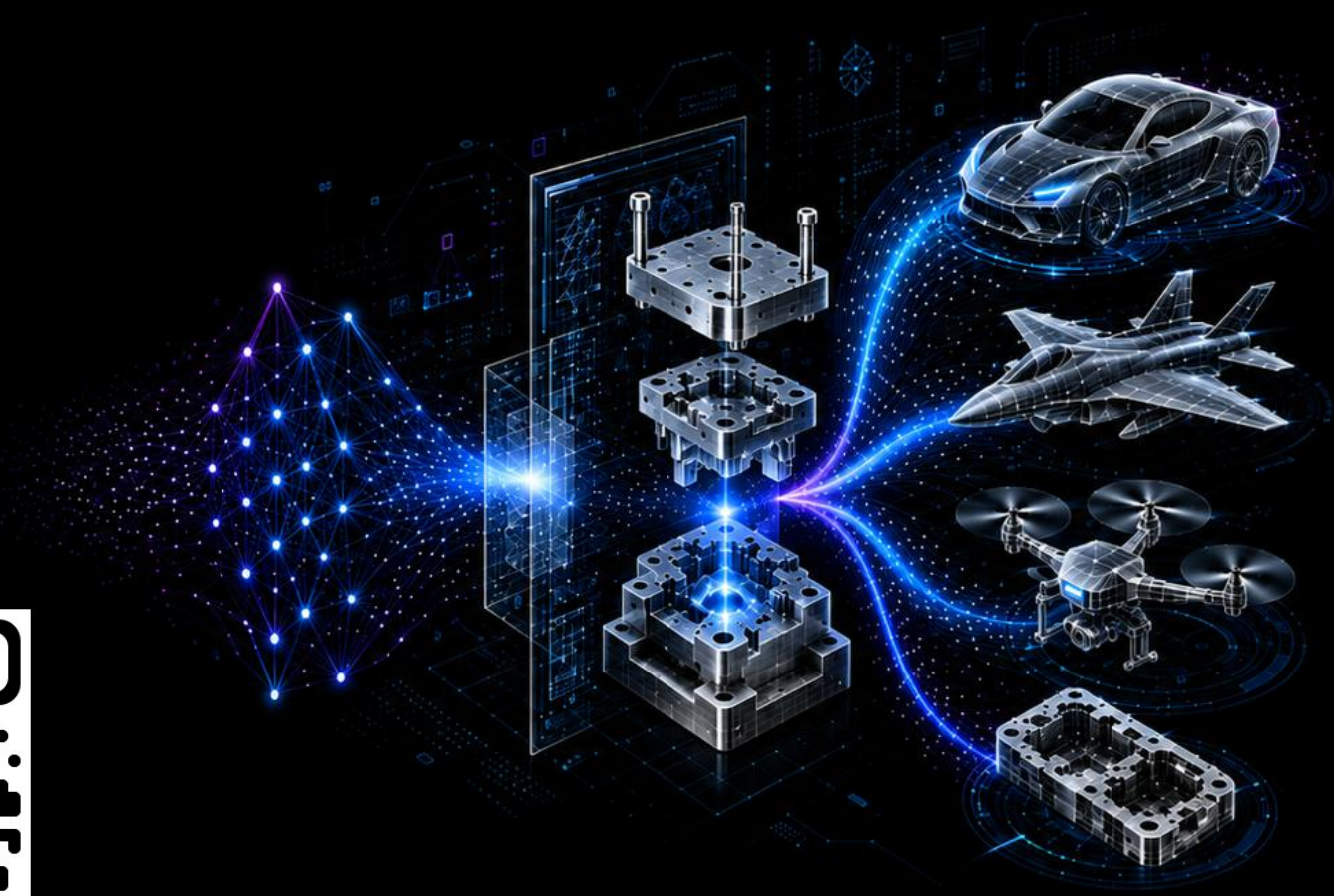
(4) $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h_t}$
새 은닉 상태, 과거와 후보의 볼록 결합

σ 는 시그모이드 함수이며 출력은 0 과 1 사이, ⊙ 는 아다마르 곱
W 와 U 는 가중치 행렬, b 는 편향 벡터, x 는 입력, h 는 은닉 상태

Figure2



Any Questions?



이름 정 승 한

소속 한양대학교 기계공학과

이메일 jshan0407@hanyang.ac.kr

Intelligent Design for Engineering Applications Lab